

Part 4: Bad Design Example

Study the following **poorly designed** code and identify what's wrong:

```
class Animal {
    String name;
    void makeSound() {
        System.out.println("Generic animal sound");
    }
}

class RobotDog extends Animal {
    int batteryLevel;
    void makeSound(String sound) { // Overloading instead of overriding
        System.out.println(sound);
    }
}
```

What are the design flaws here ?

In class animal the attribute name if it were private is better.

Is better using constructor in two classes and keyword super.

Is better using setter and getter in two classes.

The method makeSound should is overriding not overloading.

Why is using method overloading instead of overriding problematic ?

Failure to achieve the goal of using Polymorphism when make overloading the method in sub-class become different method in super-class .

How can you fix the inheritance and method structure ?

In class animal method makeSound delete parameter in the method become -> void makeSound () { // }



Part 5: Experiments with Type Assignments

Try the following code snippets and **observe the results**:

```
RobotDog dog = new Animal();           // ?  
Animal dog2 = new RobotDog();          // ✓  
Rechargeable dog3 = new RobotDog();    // ✓ (if Rechargeable is an interface)
```

Which lines compile successfully? Why or why not ?

RobotDog dog = new Animal(); // Error because the class Animal parent class RobotDog , class RobotDog Inherited from class Animal not opposite also class Animal is abstract can not make object type Animal.

Animal dog2 = new RobotDog(); // Successful because the class Animal parent class RobotDog , class RobotDog Inherited from class Animal .

Rechargeable dog3 = new RobotDog(); // Successful because the class Rechargeable interface should define object from class Create an interface from class Rechargeable .

What methods can each variable access ?

Variable dog -> nothing this is error .

Variable dog2 -> method in the class Animal .

Variable dog3 -> method in the class Rechargeable .

What happens when you call dog2.makeSound()?

Calling method makeSound from class RobotDog because overriding .

How does polymorphism affect method calls?

Polymorphism make java choose method and depending on real object not the variable type like if variable is animal but object is robotDog Java use robotDog.



Part 6: Override Behavior Exploration

```
Animal a = new RobotDog("Bolt", 2, 90);  
a.eat();  
a.makeSound();
```

Which version of eat() is executed — Animal or RobotDog ?

RobotDog .

Why does Java choose that version at runtime ?

because method eat() overridden from class RobotDog and object -> RobotDog is not type variable .

What happens if makeSound() is not overridden in RobotDog ?

Call method makeSound() in the parent class.

How does abstraction improve flexibility when adding new types of animals (e.g., Bird, Cat, RobotCat) ?

A new class only needs to inherit from Animal and override the abstract method without needing to change the existing code .

What would happen if Animal were not abstract ?

I can create an animal object, and this is wrong because the Animal class only contains general things about animals .

Why is it better to depend on abstractions (interfaces, abstract classes) rather than concrete implementations ?

The code is more flexible. I can add an object without modifying the main code .

